

Assignment 4 - CP363

Moosa Ahmed, Muhammad Saim Bin Asif

169046796, 169075698

Group # 26

Equipment Test Center

Dr. Haytham Qushtom

Due: November 2nd, 2025

Tables with Sample Data

	Equipment_ID	Name	Model_Number	Manufacturer	Status
▶	1	Centrifuge	CF-200	LabTech	Available
	2	Spectrometer	SP-900	SpecCorp	Under Maintenance
	3	Microscope	MS-1000	Optix	In Use
✱	NULL	NULL	NULL	NULL	NULL

	Client_ID	First_Name	Last_Name	Contact_Email	Phone_Number	Organization
▶	1	Aisha	Khan	aisha.khan@biotech.ca	416-555-1010	BioNova Labs
	2	Michael	Osei	m.osei@chemcore.com	416-555-1020	ChemCore Analytics
	3	Sophia	Martinez	s.martinez@greenearth.org	416-555-1030	GreenEarth Research
✱	NULL	NULL	NULL	NULL	NULL	NULL

	Technician_ID	First_Name	Last_Name	Certification	Contact_Email	Phone_Number
▶	1	Liam	Chen	ISO-15189	liam.chen@labtech.ca	647-555-1001
	2	Fatima	Rahman	ASTM-Cert	fatima.r@qualitylabs.com	647-555-1002
	3	Ethan	Brown	ISO-9001	ethan.brown@microlabs.ca	647-555-1003
*	NULL	NULL	NULL	NULL	NULL	NULL

	Test_ID	Test_Name	Description
▶	1	Water Purity	Chemical and biological analysis of water samples
	2	Microbial Culture	Culture and count of microorganisms
	3	Material Strength	Tensile and compression strength test
•	NULL	NULL	NULL

Result Grid		 Filter Rows:	Edit: 	
	Lab_ID	Lab_Name	Location	Capacity
▶	1	Main Lab	Building A - Floor 1	10
	2	Microbiology Lab	Building C - Floor 3	8
	3	Materials Lab	Building B - Floor 2	6
*	NULL	NULL	NULL	NULL

[illegible]

	Session_ID	Test_Date	Result	Duration	Technician_ID	Test_ID
▶	1	2025-10-05	Pass	01:30:00	1	1
	2	2025-10-06	Fail	02:00:00	2	2
	3	2025-10-07	Pending	01:45:00	3	3
★	NULL	NULL	NULL	NULL	NULL	NULL

	Report_ID	Report_Date	Findings	Approved_By	Session_ID
▶	1	2025-10-05	Sample meets purity standards	1	1
	2	2025-10-06	Microbial count exceeded limits	2	2
	3	2025-10-07	Testing in progress	3	3
★	NULL	NULL	NULL	NULL	NULL

	Invoice_ID	Booking_ID	Invoice_Date	Amount	Payment_Status
▶	1	1	2025-10-05	250.00	Paid
	2	2	2025-10-06	180.00	Unpaid
	3	3	2025-10-07	200.00	Unpaid
★	NULL	NULL	NULL	NULL	NULL

	Booking_ID	Equipment_ID
▶	1	1
	2	2
	3	3
★	NULL	NULL

	Session_ID	Equipment_ID
▶	1	1
	2	2
	3	3
★	NULL	NULL

	Maintenance_ID	Equipment_ID	Maintenance_Date	Technician_ID	Notes
▶	1	1	2025-09-10	1	Rotor inspected and cleaned
	2	2	2025-09-15	2	Lamp replaced and calibrated
	3	3	2025-09-20	3	Lens alignment adjusted
★	NULL	NULL	NULL	NULL	NULL

7 Advanced queries

Advanced query 1: Per-client invoice statistics

```
91 • SELECT
92     c.Client_ID,
93     CONCAT(c.First_Name, ' ', c.Last_Name) AS Client_Name,
94     COUNT(DISTINCT b.Booking_ID) AS Num_Bookings,
95     COUNT(i.Invoice_ID) AS Num_Invoices,
96     COALESCE(SUM(i.Amount), 0.00) AS Total_Invoiced,
97     COALESCE(AVG(i.Amount), 0.00) AS Avg_Invoice,
98     COALESCE(STDDEV_POP(i.Amount), 0.00) AS Invoice_StdDev
99 FROM Client c
100 LEFT JOIN Booking b ON b.Client_ID = c.Client_ID
101 LEFT JOIN Invoice i ON i.Booking_ID = b.Booking_ID
102 GROUP BY c.Client_ID
103 ORDER BY Total_Invoiced DESC;
104
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Client_ID	Client_Name	Num_Bookings	Num_Invoices	Total_Invoiced	Avg_Invoice	Invoice_StdDev
▶	1	Aisha Khan	1	1	250.00	250.000000	0
	3	Sophia Martinez	1	1	200.00	200.000000	0
	2	Michael Osei	1	1	180.00	180.000000	0

Advanced query 2: Equipment used in bookings OR test sessions (UNION)

```
105 • SELECT e.Equipment_ID, e.Name, 'Booking' AS Source
106 FROM Equipment e
107 JOIN Booking_Equipment be ON be.Equipment_ID = e.Equipment_ID
108
109 UNION
110
111 SELECT e.Equipment_ID, e.Name, 'TestSession' AS Source
112 FROM Equipment e
113 JOIN TestSession_Equipment tse ON tse.Equipment_ID = e.Equipment_ID
114 ORDER BY Equipment_ID;
115
```

Result Grid

 Filter Rows:

Export: 

Wrap Cell Content: 

	Equipment_ID	Name	Source
▶	1	Centrifuge	Booking
	1	Centrifuge	TestSession
	2	Spectrometer	Booking
	2	Spectrometer	TestSession
	3	Microscope	Booking
	3	Microscope	TestSession

Advanced query 3: Equipment in bookings but never used in test sessions (Anti-join)

```

119      # Advanced Query 3:
120      SELECT DISTINCT E.Equipment_ID,
121                      E.Name,
122                      E.Model_Number
123      FROM Equipment E
124      JOIN Booking_Equipment BE
125           ON E.Equipment_ID = BE.Equipment_ID
126      LEFT JOIN TestSession_Equipment TSE
127           ON E.Equipment_ID = TSE.Equipment_ID
128      WHERE TSE.Equipment_ID IS NULL;
129
130      # Advanced Query 4:

```

Result Grid
Filter Rows:
Export:
Wrap Cell

Equipment_ID	Name	Model_Number
--------------	------	--------------

Advanced query 4: Rank tests by average session duration (Window function)

```

130      # Advanced Query 4:
131      SELECT
132          t.Test_ID,
133          t.Test_Name,
134          COUNT(ts.Session_ID) AS Sessions,
135          ROUND(AVG(TIME_TO_SEC(ts.Duration)) / 60, 2) AS Avg_Duration_Min,
136          RANK() OVER (ORDER BY AVG(TIME_TO_SEC(ts.Duration)) DESC) AS Duration_Rank
137      FROM Test t
138      LEFT JOIN Test_Session ts ON ts.Test_ID = t.Test_ID
139      GROUP BY t.Test_ID;
140

```

Result Grid
Filter Rows:
Export:
Wrap Cell Content:

	Test_ID	Test_Name	Sessions	Avg_Duration_Min	Duration_Rank
▶	2	Microbial Culture	1	120.00	1
	3	Material Strength	1	105.00	2
	1	Water Purity	1	90.00	3

Advanced query 5: Technicians with failure rate of test sessions

```
141 # Advanced Query 5:
142 • SELECT
143     tech.Technician_ID,
144     CONCAT(tech.First_Name, ' ', tech.Last_Name) AS Technician,
145     COUNT(ts.Session_ID) AS Total_Sessions,
146     SUM(CASE WHEN ts.Result = 'Fail' THEN 1 ELSE 0 END) AS Failed_Sessions,
147     ROUND(
148         SUM(CASE WHEN ts.Result = 'Fail' THEN 1 ELSE 0 END) / GREATEST(COUNT(ts.Session_ID), 1) * 100, 2
149     ) AS Fail_Pct
150 FROM Technician tech
151 LEFT JOIN Test_Session ts ON ts.Technician_ID = tech.Technician_ID
152 GROUP BY tech.Technician_ID
153 HAVING Total_Sessions > 0 AND Fail_Pct > 30
154 ORDER BY Fail_Pct DESC;
155
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Technician_ID	Technician	Total_Sessions	Failed_Sessions	Fail_Pct
	2	Fatima Rahman	1	1	100.00

Advanced query 6: Latest maintenance per equipment

```
156 # Advanced Query 6:
157 • SELECT
158     e.Equipment_ID,
159     e.Name,
160     e.Status,
161     (SELECT MAX(m.Maintenance_Date) FROM Maintenance_Record m WHERE m.Equipment_ID = e.Equipment_ID) AS Last_Maintenance_Date,
162     (SELECT CONCAT(t.First_Name, ' ', t.Last_Name)
163     FROM Maintenance_Record m
164     JOIN Technician t ON t.Technician_ID = m.Technician_ID
165     WHERE m.Equipment_ID = e.Equipment_ID
166     ORDER BY m.Maintenance_Date DESC
167     LIMIT 1
168 ) AS Last_Maintained_By
169 FROM Equipment e;
170
171
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content: [↕](#)

	Equipment_ID	Name	Status	Last_Maintenance_Date	Last_Maintained_By
▶	1	Centrifuge	Available	2025-09-10	Liam Chen
	2	Spectrometer	Under Maintenance	2025-09-15	Fatima Rahman
	3	Microscope	In Use	2025-09-20	Ethan Brown

Advanced query 7: 14 day calendar of bookings (Recursive)

```

172 # Advanced Query 7:
173 WITH RECURSIVE calendar AS (
174     SELECT CURDATE() AS cal_date, 1 AS day_num
175     UNION ALL
176     SELECT DATE_ADD(cal_date, INTERVAL 1 DAY), day_num + 1
177     FROM calendar
178     WHERE day_num < 14
179 )
180 SELECT
181     cal.cal_date,
182     COUNT(b.Booking_ID) AS Num_Bookings,
183     COALESCE(GROUP_CONCAT(CONCAT(b.Booking_ID, ': ', c.First_Name, ' ', c.Last_Name) SEPARATOR '; '), '') AS Bookings_Detail
184 FROM calendar cal
185 LEFT JOIN Booking b ON b.Booking_Date = cal.cal_date
186 LEFT JOIN Client c ON c.Client_ID = b.Client_ID
187 GROUP BY cal.cal_date
188 ORDER BY cal.cal_date;

```

189

Result Grid			
Filter Rows:		Export:	Wrap Cell Content:
cal_date	Num_Bookings	Bookings_Detail	
2025-11-01	0		
2025-11-02	0		
2025-11-03	0		
2025-11-04	0		
2025-11-05	0		
2025-11-06	0		
2025-11-07	0		
2025-11-08	0		
2025-11-09	0		
2025-11-10	0		
2025-11-11	0		
2025-11-12	0		
2025-11-13	0		
2025-11-14	0		

Execution plan analysis & Optimization

We analyzed **Advanced Query 1** (Client Invoice Summary).

```
91 • SELECT
92     c.Client_ID,
93     CONCAT(c.First_Name, ' ', c.Last_Name) AS Client_Name,
94     COUNT(DISTINCT b.Booking_ID)          AS Num_Bookings,
95     COUNT(i.Invoice_ID)                   AS Num_Invoices,
96     COALESCE(SUM(i.Amount), 0.00)        AS Total_Invoiced,
97     COALESCE(AVG(i.Amount), 0.00)        AS Avg_Invoice,
98     COALESCE(STDDEV_POP(i.Amount), 0.00) AS Invoice_StdDev
99 FROM Client c
100 LEFT JOIN Booking b ON b.Client_ID = c.Client_ID
101 LEFT JOIN Invoice i ON i.Booking_ID = b.Booking_ID
102 GROUP BY c.Client_ID
103 ORDER BY Total_Invoiced DESC;
104
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 							
	Client_ID	Client_Name	Num_Bookings	Num_Invoices	Total_Invoiced	Avg_Invoice	Invoice_StdDev
▶	1	Aisha Khan	1	1	250.00	250.000000	0
	3	Sophia Martinez	1	1	200.00	200.000000	0
	2	Michael Osei	1	1	180.00	180.000000	0

Execution Plan Summary

- MySQL performs joins between Client, Booking and Invoice.
- The most expensive part is aggregating values from Invoice.Amount (SUM, AVG, STDDEV).
- The query requires MySQL to scan invoice rows for each client to compute totals.

Optimization Suggestion

Add an index on the Invoice.Amount column to speed up aggregate operations. This improves performance when invoice table grows large.

- CREATE INDEX idx_invoice_amount ON Invoice(Amount);

3 Advanced views

Advanced view 1: A view with subquery in the SELECT clause to compute a derived value

```
205 # 3 Advanced Views:
206 # Advanced View 1
207 • CREATE OR REPLACE VIEW Client_LatestBooking AS
208 SELECT
209     c.Client_ID,
210     CONCAT(c.First_Name, ' ', c.Last_Name) AS Client_Name,
211     (SELECT MAX(b.Booking_Date)
212      FROM Booking b
213      WHERE b.Client_ID = c.Client_ID) AS Latest_Booking_Date
214 FROM Client c;
215
216 • SELECT * FROM Client_LatestBooking;
```

Client_ID	Client_Name	Latest_Booking_Date
1	Aisha Khan	2025-10-05
2	Michael Osei	2025-10-06
3	Sophia Martinez	2025-10-07

Advanced view 2: A view using subquery in the FROM clause.

```
218 # Advanced View 2
219 • CREATE OR REPLACE VIEW Client_InvoiceSummary AS
220 SELECT
221     ci.Client_ID,
222     ci.Client_Name,
223     ci.Total_Invoiced,
224     ci.Num_Invoices
225 FROM (
226     SELECT
227         c.Client_ID,
228         CONCAT(c.First_Name, ' ', c.Last_Name) AS Client_Name,
229         COALESCE(SUM(i.Amount), 0) AS Total_Invoiced,
230         COUNT(i.Invoice_ID) AS Num_Invoices
231     FROM Client c
232     LEFT JOIN Booking b ON c.Client_ID = b.Client_ID
233     LEFT JOIN Invoice i ON b.Booking_ID = i.Booking_ID
234     GROUP BY c.Client_ID
235 ) ci;
236
237 • SELECT * FROM Client_InvoiceSummary;
```

Client_ID	Client_Name	Total_Invoiced	Num_Invoices
1	Aisha Khan	250.00	1
2	Michael Osei	180.00	1
3	Sophia Martinez	200.00	1

Advanced view 3: A view using subquery in the WHERE clause to filter data dynamically.

```
239 # Advanced View 3
240 • CREATE OR REPLACE VIEW TopTechnician_Sessions AS
241     SELECT ts.Session_ID,
242            ts.Test_Date,
243            ts.Result,
244            ts.Technician_ID,
245            CONCAT(t.First_Name, ' ', t.Last_Name) AS Technician_Name
246     FROM Test_Session ts
247     JOIN Technician t ON ts.Technician_ID = t.Technician_ID
248     WHERE ts.Technician_ID = (
249         SELECT Technician_ID
250        FROM Test_Session
251       GROUP BY Technician_ID
252      ORDER BY COUNT(*) DESC
253      LIMIT 1
254    );
255
256 • SELECT * FROM TopTechnician_Sessions;
257
```

Result Grid					
Filter Rows: Export: Wrap Cell Content:					
	Session_ID	Test_Date	Result	Technician_ID	Technician_Name
▶	1	2025-10-05	Pass	1	Liam Chen

Python code

```
12 import mysql.connector
13 from mysql.connector import errorcode
14
15 # MySQL Connection
16 try:
17     conn = mysql.connector.connect(
18         host="localhost",
19         port = 3306,
20         user = "root",
21         password="36990242@MySQL",
22         database="EquipmentTestCenterDB"
23     )
24     cursor = conn.cursor(dictionary=True)
25 except mysql.connector.Error as err:
26     if err.errno == errorcode.ER_BAD_DB_ERROR:
27         print("Database does not exist.")
28     else:
29         print(err)
30     exit()
31
32 # menu
33 def menu():
34     print("\n===== Equipment Test Center Menu =====")
35     print("1. Show all clients")
36     print("2. Show all equipment currently in use")
37     print("3. Show clients with confirmed bookings")
38     print("4. Show invoices above average")
39     print("5. Show equipment booked but never used")
40     print("6. Exit")
41
42 # Queries
43 def show_clients():
44     cursor.execute("SELECT * FROM Client;")
45     rows = cursor.fetchall()
46     for row in rows:
47         print(row)
48
49 def equipment_in_use():
50     cursor.execute("SELECT Equipment_ID, Name, Status FROM Equipment WHERE Status='In Use';")
51     rows = cursor.fetchall()
52     for row in rows:
53         print(row)
54
```

```

55 def clients_confirmed_bookings():
56     cursor.execute("""
57         SELECT First_Name, Last_Name
58         FROM Client
59         WHERE Client_ID IN (
60             SELECT Client_ID
61             FROM Booking
62             WHERE Status='Confirmed'
63         );
64     """)
65     rows = cursor.fetchall()
66     for row in rows:
67         print(row)
68
69 def invoices_above_avg():
70     cursor.execute("""
71         SELECT i.Invoice_ID, i.Booking_ID, i.Amount
72         FROM Invoice i
73         JOIN (SELECT AVG(Amount) AS Avg_Amount FROM Invoice) avg_table
74         ON i.Amount > avg_table.Avg_Amount;
75     """)
76     rows = cursor.fetchall()
77     for row in rows:
78         print(row)
79
80 def equipment_booked_never_used():
81     cursor.execute("""
82         SELECT DISTINCT E.Equipment_ID, E.Name, E.Model_Number
83         FROM Equipment E
84         JOIN Booking_Equipment BE ON E.Equipment_ID = BE.Equipment_ID
85         LEFT JOIN TestSession_Equipment TSE ON E.Equipment_ID = TSE.Equipment_ID
86         WHERE TSE.Equipment_ID IS NULL;
87     """)
88     rows = cursor.fetchall()
89     if not rows:
90         print("All booked equipment has been used in test sessions.")
91     else:
92         for row in rows:
93             print(row)
94

```

```

94
95 # Main Loop
96 while True:
97     menu()
98     choice = input("Enter your choice (1-6): ")
99
100     if choice == '1':
101         show_clients()
102     elif choice == '2':
103         equipment_in_use()
104     elif choice == '3':
105         clients_confirmed_bookings()
106     elif choice == '4':
107         invoices_above_avg()
108     elif choice == '5':
109         equipment_booked_never_used()
110     elif choice == '6':
111         print("Exiting...")
112         break
113     else:
114         print("Invalid choice. Try again.")
115
116 cursor.close()
117 conn.close()

```

Functional dependencies

1. Equipment Table

- **FDs:** Equipment_ID → Name, Model_Number, Manufacturer, Status
- **Impact:**
 - Queries filtering by Equipment_ID are fast due to primary key indexing.
 - Storing status in the same table avoids joins to check availability.
- **Normalization:** 3NF; all non-key attributes depend only on Equipment_ID.

2. Client Table

- **FDs:** Client_ID → First_Name, Last_Name, Contact_Email, Phone_Number, Organization
Contact_Email → Client_ID and Phone_Number → Client_ID (since they are UNIQUE)
- **Impact:**
 - Ensures fast lookups using email or phone.
 - Joins with Booking or Invoice use Client_ID.
- **Normalization:** 3NF; client info not repeated elsewhere.

3. Technician Table

- **FDs:** Technician_ID → First_Name, Last_Name, Certification, Contact_Email, Phone_Number
Contact_Email → Technician_ID and Phone_Number → Technician_ID (UNIQUE)
- **Impact:**
 - Supports efficient joins with **Test_Session** and **Maintenance_Record**.
- **Normalization:** 3NF; prevents duplication in multiple sessions.

4. Test Table

- **FDs:** Test_ID → Test_Name, Description
- **Impact:**
 - Fast lookup for sessions by test ID.
- **Normalization:** 3NF; avoids repeating test names and descriptions.

5. Lab Table

- **FDs:** Lab_ID → Lab_Name, Location, Capacity
- **Impact:**
 - Joins with Booking to quickly check capacity and lab info.
- **Normalization:** 3NF; no redundant info in bookings.

6. Booking Table

- **FDs:** Booking_ID → Booking_Date, Client_ID, Lab_ID, Start_Time, End_Time, Status
(Client_ID, Booking_Date, Lab_ID) → Booking_ID (assuming no double booking in same lab/date)
- **Impact:**
 - Supports aggregation per client or per lab efficiently.
 - Indexing Client_ID or Lab_ID helps fast filtering.
- **Normalization:** 3NF; avoids repeating client or lab details in each booking.

7. Test_Session Table

- **FDs:** Session_ID → Test_Date, Result, Duration, Technician_ID, Test_ID
- **Impact:**
 - Fast retrieval of sessions by test or technician.
 - Aggregation (e.g., average duration) is easy.
- **Normalization:** 3NF; no redundant storage of technician or test info.

8. Report Table

- **FDs:** Report_ID → Report_Date, Findings, Approved_By, Session_ID
Session_ID → Report_ID (UNIQUE)
- **Impact:**
 - Ensures only one report per session.
 - Efficient join with Technician for report approval info.
- **Normalization:** 3NF; no duplicate report data.

9. Invoice Table

- **FDs:** Invoice_ID → Booking_ID, Invoice_Date, Amount, Payment_Status
Booking_ID → Invoice_ID (assuming one invoice per booking)
- **Impact:**
 - Aggregation per client (sum of invoices) is straightforward.
 - Joins with Booking and Client are optimized with indexes.
- **Normalization:** 3NF; avoids storing booking details multiple times.

10. Maintenance_Record Table

- **FDs:** Maintenance_ID → Equipment_ID, Maintenance_Date, Technician_ID, Notes
- **Impact:**
 - Fast queries for last maintenance of equipment using Equipment_ID index.
- **Normalization:** 3NF; technician and equipment info not repeated.

11. Booking_Equipment Table (Junction Table)

- **FDs:** (Booking_ID, Equipment_ID) → nothing else
- **Impact:**
 - Supports many-to-many relationship efficiently.
Indexes on Equipment_ID or Booking_ID speed up anti-joins and availability checks.
- **Normalization:** Fully normalized; avoids redundant equipment lists.

12. TestSession_Equipment Table (Junction Table)

- **FDs:** (Session_ID, Equipment_ID) → nothing else
- **Impact:**
 - Efficient joins to find which equipment was used in sessions.
 - Supports anti-joins for “booked but never used” queries.
- **Normalization:** Fully normalized; avoids duplicating equipment usage info.